

KHAN INSTITUTE OF
COMPUTER SCIENCE AND
INFORMATION TECHNOLOGY
COMPUTER SCIENCE DEPARTMENT

Open-Ended Lab

Library Management System

Submitted by:

Muhammad Bilal

232201100

Submitted to:

Sir Uzair Hassan

Department of Computer Science

May 7, 2026

Contents

1	Problem Description	2
2	Objectives	2
3	System Design	2
3.1	Architecture Overview	2
3.2	Class Diagram (OOP Design)	3
3.3	Database Schema	3
4	Implementation Details	3
4.1	OOP Principles Applied	3
4.2	REST API Integration — Open Library	4
4.3	Database Integration — Entity Framework Core	4
5	Docker Setup	5
5.1	Dockerfile — Multi-Stage Build	5
5.2	Docker Compose	6
5.3	Running the Application	6
6	Results & Discussion	6
6.1	Functional Features	6
6.2	OOP Coverage Summary	7
6.3	Challenges & Solutions	7
7	Conclusion	7

1. Problem Description

Libraries manage large collections of books, members, and loan records. Without a proper system, tracking which books are available, who has borrowed them, and when they are due becomes error-prone and time-consuming.

BookVault is a web-based Library Management System that digitizes these operations. It allows library staff to:

- Add, edit, search, and delete books with automatic cover image retrieval
- Register and manage library members
- Issue books to members and process returns
- Automatically calculate overdue fines
- Secure access using a login/register system

The system uses **Open Library REST API** to automatically fetch book cover images by ISBN, demonstrating real-world API integration.

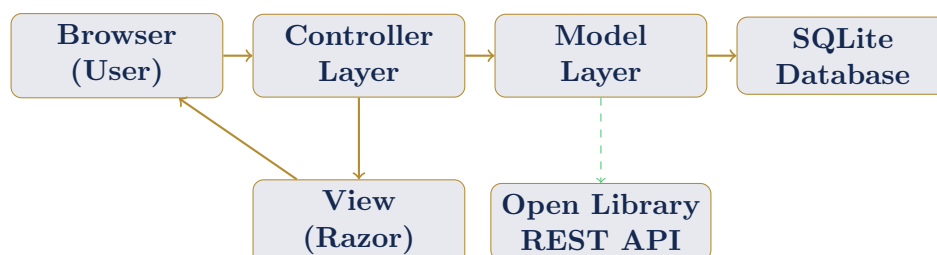
2. Objectives

1. Apply OOP principles — Inheritance, Polymorphism, Encapsulation, Abstraction
2. Build a functional web UI using ASP.NET Core MVC (Razor Views)
3. Implement full CRUD operations with SQLite via Entity Framework Core
4. Integrate a REST API (Open Library) to fetch book cover images dynamically
5. Containerize and deploy the application using Docker and Docker Compose
6. Implement user authentication using ASP.NET Core Identity

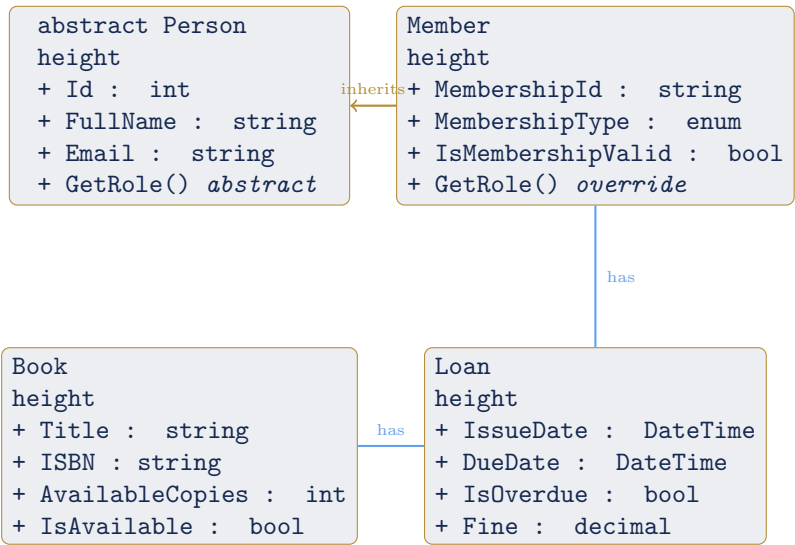
3. System Design

3.1. Architecture Overview

The application follows the **MVC (Model-View-Controller)** pattern:



3.2. Class Diagram (OOP Design)



3.3. Database Schema

Table	Column	Description
Books	Id, Title, Author, ISBN Genre, Publisher, Year TotalCopies, AvailableCopies CoverImageUrl	Core book information Classification details Inventory tracking From Open Library API
Members	Id, FullName, Email MembershipId, MembershipType MembershipExpiry, IsActive	Identity information Library-specific fields Status tracking
Loans	Id, BookId, MemberId IssueDate, DueDate, ReturnDate IsReturned	FK relationships Date tracking Status flag

4. Implementation Details

4.1. OOP Principles Applied

1. Inheritance — Member inherits from abstract base class Person:

```
1 public abstract class Person
2 {
3     public int Id { get; set; }
4     public string FullName { get; set; } = "";
5     public string Email { get; set; } = "";
6
7     // Abstract method - forces subclass to implement
8     public abstract string GetRole();
9 }
10
11 public class Member : Person
```

```

12 {
13     public string MembershipId { get; set; } = "";
14     public MembershipType MembershipType { get; set; }
15
16     // POLYMORPHISM - override abstract method
17     public override string GetRole() => $"Member_{MembershipType}";
18
19     // ENCAPSULATION - computed property
20     public bool IsMembershipValid =>
21         IsActive && MembershipExpiry >= DateTime.Now;
22 }

```

2. Encapsulation — Business logic inside model properties:

```

1 public class Loan
2 {
3     public DateTime DueDate { get; set; }
4     public bool IsReturned { get; set; }
5
6     // Encapsulated computed properties
7     public bool IsOverdue => !IsReturned && DateTime.Now >
8         DueDate;
9     public int DaysOverdue => IsOverdue
10         ? (int)(DateTime.Now - DueDate).TotalDays :
11         0;
12     public decimal Fine => IsOverdue ? DaysOverdue * 10m : 0;
13     public string Status => IsReturned ? "Returned"
14         : IsOverdue ? "Overdue" : "Active";
15 }

```

4.2. REST API Integration — Open Library

Book cover images are fetched automatically from **Open Library API** using the ISBN. No API key is required.

```

1 // In BooksController.cs - Create action
2 if (!string.IsNullOrEmpty(book.ISBN))
3 {
4     var cleanISBN = book.ISBN.Replace("-", "");
5     book.CoverImageUrl =
6         $"https://covers.openlibrary.org/b/isbn/{cleanISBN}-M.jpg";
7 }

```

In the Razor View, the image loads from the URL and falls back gracefully:

```

1 

```

4.3. Database Integration — Entity Framework Core

CRUD operations are implemented in all controllers. Example from `BooksController.cs`:

```
1 // READ - with search and filter
2 public async Task<IActionResult> Index(string? search, string?
   genre)
3 {
4     var query = _db.Books.AsQueryable();
5     if (!string.IsNullOrEmpty(search))
6         query = query.Where(b => b.Title.Contains(search)
7                               || b.Author.Contains(search));
8     return View(await query.OrderBy(b => b.Title).ToListAsync());
9 }
10
11 // CREATE
12 public async Task<IActionResult> Create(Book book)
13 {
14     _db.Books.Add(book);
15     await _db.SaveChangesAsync();
16     return RedirectToAction(nameof(Index));
17 }
18
19 // DELETE - with active loan check
20 public async Task<IActionResult> DeleteConfirmed(int id)
21 {
22     var book = await _db.Books.FindAsync(id);
23     if (book.AvailableCopies < book.TotalCopies)
24     {
25         TempData["Error"] = "Cannot delete - book has active loans!";
26         return RedirectToAction(nameof(Index));
27     }
28     _db.Books.Remove(book);
29     await _db.SaveChangesAsync();
30     return RedirectToAction(nameof(Index));
31 }
```

5. Docker Setup

5.1. Dockerfile — Multi-Stage Build

A multi-stage Dockerfile keeps the final image small by separating the SDK build environment from the runtime:

```
1 # Stage 1: Build using full .NET SDK
2 FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
3 WORKDIR /src
4 COPY LibraryMS.csproj .
5 RUN dotnet restore
6 COPY . .
7 RUN dotnet publish -c Release -o /app/publish
8
9 # Stage 2: Runtime - smaller final image
10 FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
11 WORKDIR /app
12 COPY --from=build /app/publish .
13 EXPOSE 8080
14 ENV ASPNETCORE_URLS=http://+:8080
15 ENTRYPOINT ["dotnet", "LibraryMS.dll"]
```

5.2. Docker Compose

```

1 services:
2   web:
3     build:
4       context: .
5       dockerfile: Dockerfile
6     container_name: bookvault-app
7     ports:
8       - "8080:8080"
9     environment:
10      - ASPNETCORE_ENVIRONMENT=Production
11     volumes:
12      - library_data:/app # persists SQLite database
13     restart: unless-stopped
14
15 volumes:
16   library_data:

```

5.3. Running the Application

```

# Build and start container
docker-compose up --build

# Access the application
http://localhost:8080

# Default credentials
Email:      admin@library.com
Password:   admin123

```

6. Results & Discussion

6.1. Functional Features

darkblue!15 Feature	Description	Status
Book Management	Add/Edit/Delete/Search books with cover images	
Member Management	Register members, assign membership types	
Loan System	Issue books, process returns, track due dates	
Fine Calculation	Auto Rs.10/day for overdue books	
API Integration	Open Library cover images via ISBN	
Authentication	Login/Register via ASP.NET Identity	
Docker Deploy	Multi-stage build + Docker Compose	
Exception Handling	Try-catch in all controllers + TempData alerts	

6.2. OOP Coverage Summary

darkblue!15 OOP Concept	Where Applied
Encapsulation	<code>IsAvailable</code> , <code>Fine</code> , <code>IsOverdue</code> , <code>Status</code> — all computed inside model classes
Inheritance	<code>Member</code> extends abstract class <code>Person</code> ; inherits <code>Id</code> , <code>FullName</code> , <code>Email</code>
Polymorphism	<code>GetRole()</code> is abstract in <code>Person</code> , overridden in <code>Member</code>
Abstraction	<code>Person</code> is abstract — cannot be instantiated directly; forces contract implementation

6.3. Challenges & Solutions

1. **RZ1031 Razor Error** — Ternary expressions (`? :`) cannot be used inside HTML tag helper attribute declarations. Resolved by replacing inline `C#` with proper `@if` / `@else` blocks.
2. **Docker Desktop** not running — The Docker daemon must be running before executing `docker-compose up`. Resolved by starting Docker Desktop first.
3. **Cascading Deletes** — Books with active loans cannot be deleted. Handled by checking `AvailableCopies < TotalCopies` before allowing deletion.

7. Conclusion

BookVault successfully demonstrates a complete, real-world application built on modern .NET technologies. The project integrates all required components:

- **OOP** — All four principles (Encapsulation, Inheritance, Polymorphism, Abstraction) are clearly demonstrated through the domain models
- **Web UI** — A professional dark-themed ASP.NET Core MVC web application with responsive design
- **Database** — Full CRUD operations using Entity Framework Core with SQLite, including relational data (`Books ↔ Members ↔ Loans`)
- **API** — Open Library REST API integrated to dynamically fetch book cover images
- **Docker** — Multi-stage Dockerfile and Docker Compose for one-command deployment
- **Security** — Authentication system using ASP.NET Core Identity

The system is production-ready, runs inside a Docker container, and provides a clean, intuitive interface for managing an entire library's operations.

ASP.NET Core 8 · Entity Framework Core · SQLite · Docker